

Unit-V

Backtracking

Backtracking: General method,
Applications: n-queen's problem,
sum of subsets problem,
graph colouring,
Hamiltonian cycles,
Maze generation Problem

Explicit constraints :

Explicit constraints are rules that restrict each x_i to take values only from a given set.

Example :

0/1 Knapsack problem, $x_i = 0$ or 1 , or $S_i = \{ 0, 1 \}$

All tuples that satisfy the explicit constraints define a ***solution space***.

0/1 Knapsack problem solution space for $n=3$.

000, 100, 001, 101, 010, 110, 011, 111

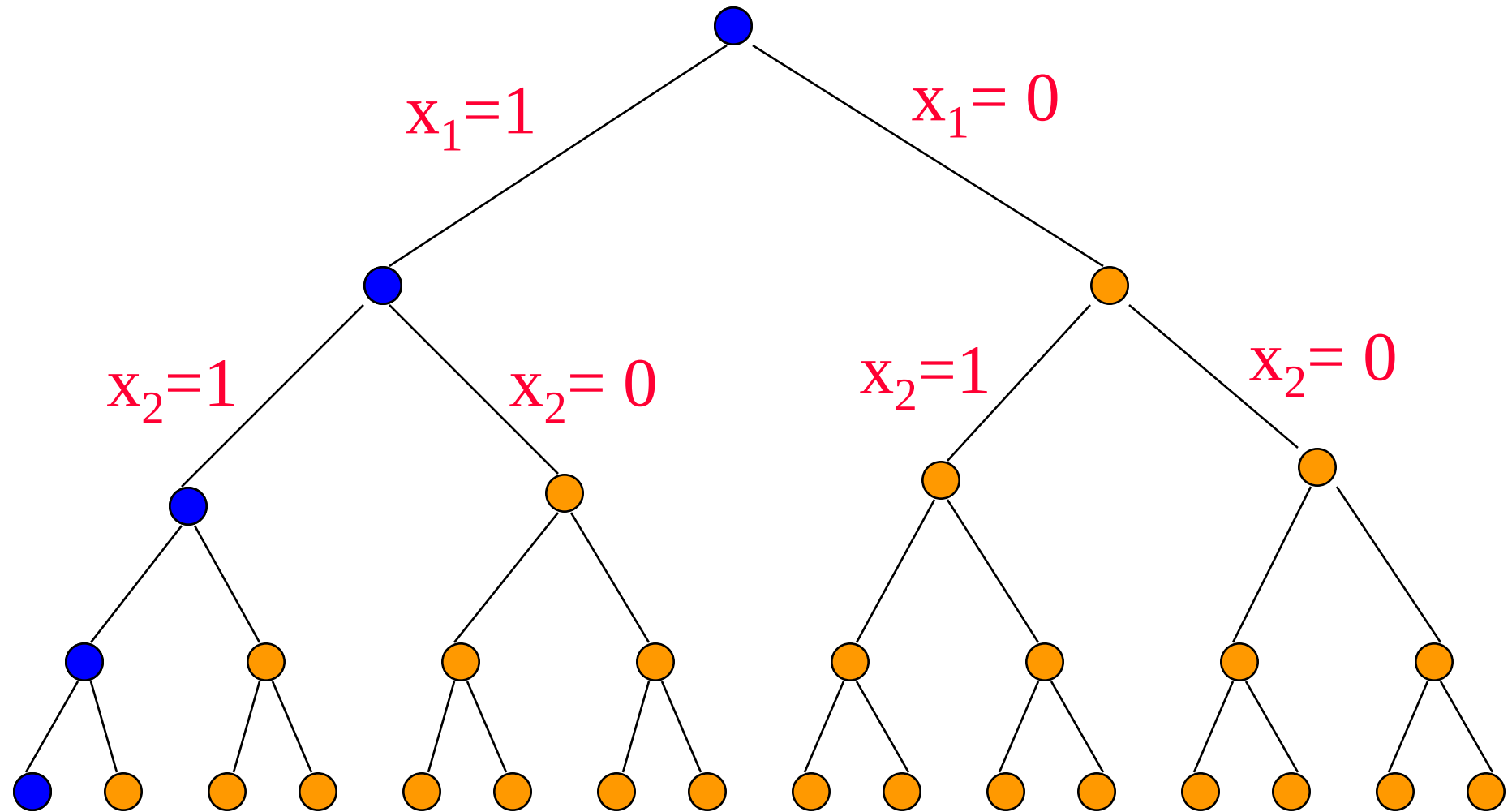
Implicit constraints:

The implicit constraints are rules that determine which of the tuples in the solution space satisfy the given conditions (criterion function) (***Answers to the given problem***)

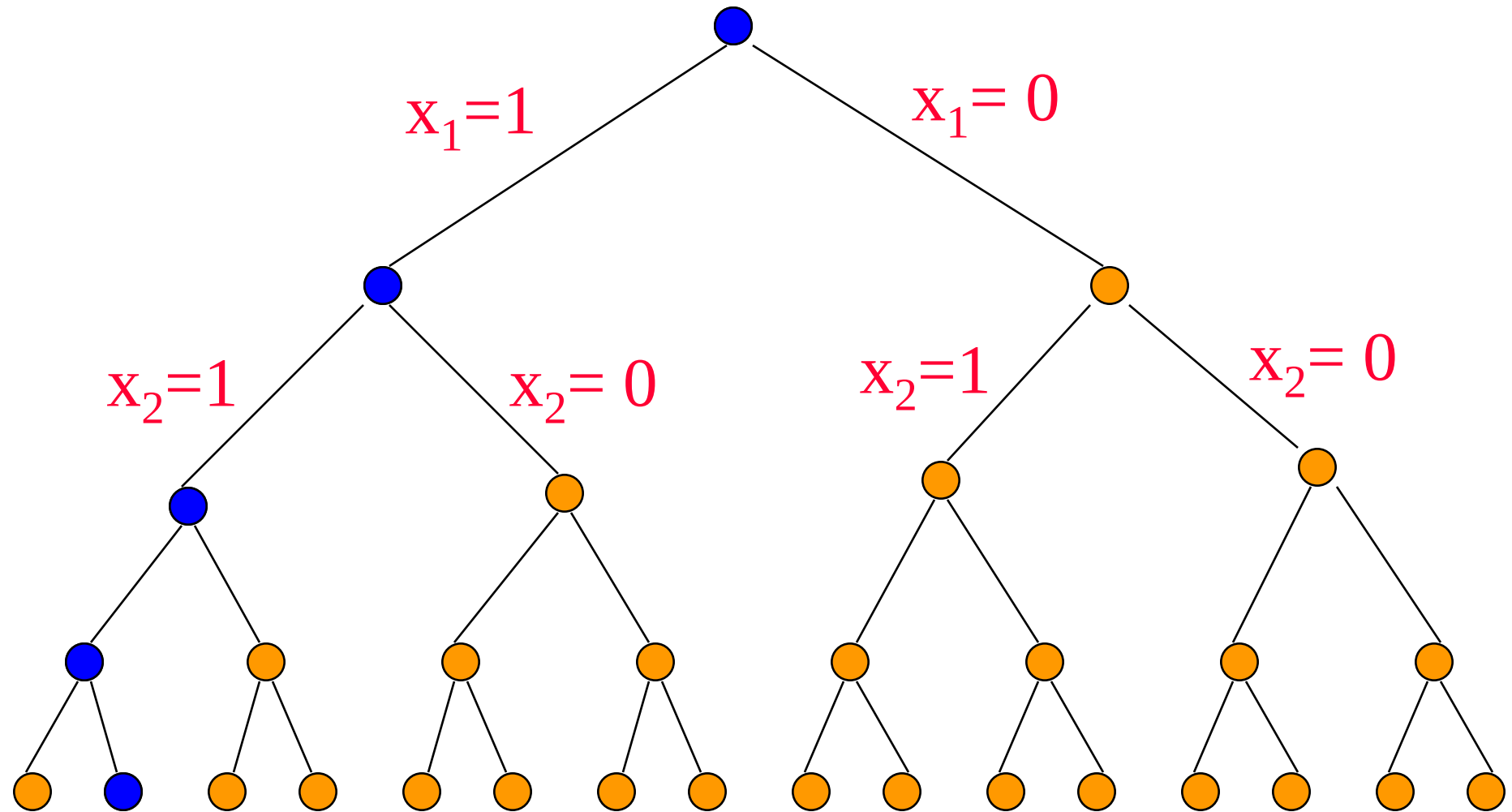
Backtracking

- Searches the solution space tree in a *depth-first* manner.
- Will be done *recursively*.
- The *solution space tree* exists only in your mind, not in the computer.

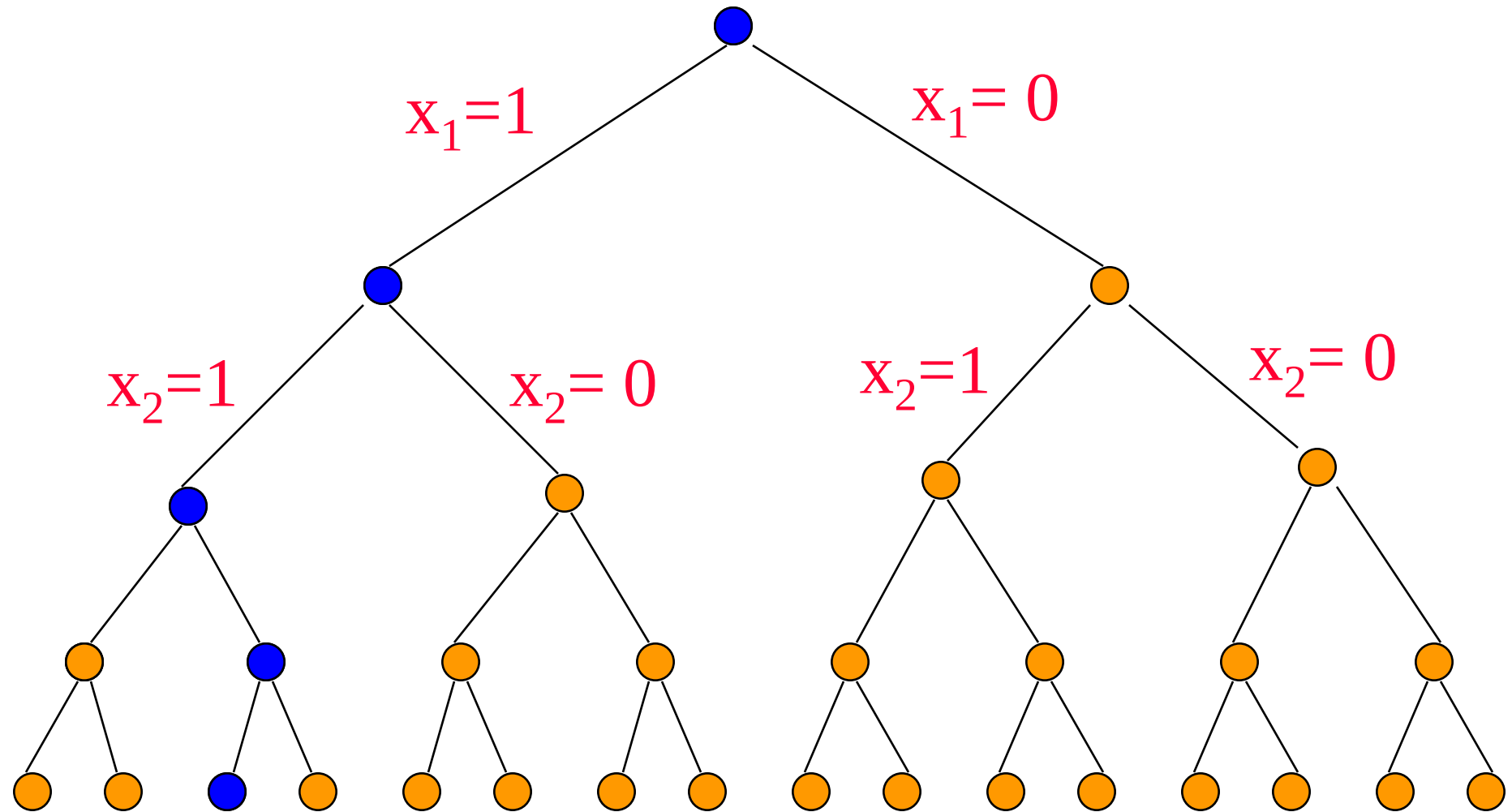
Backtracking Depth-First Search



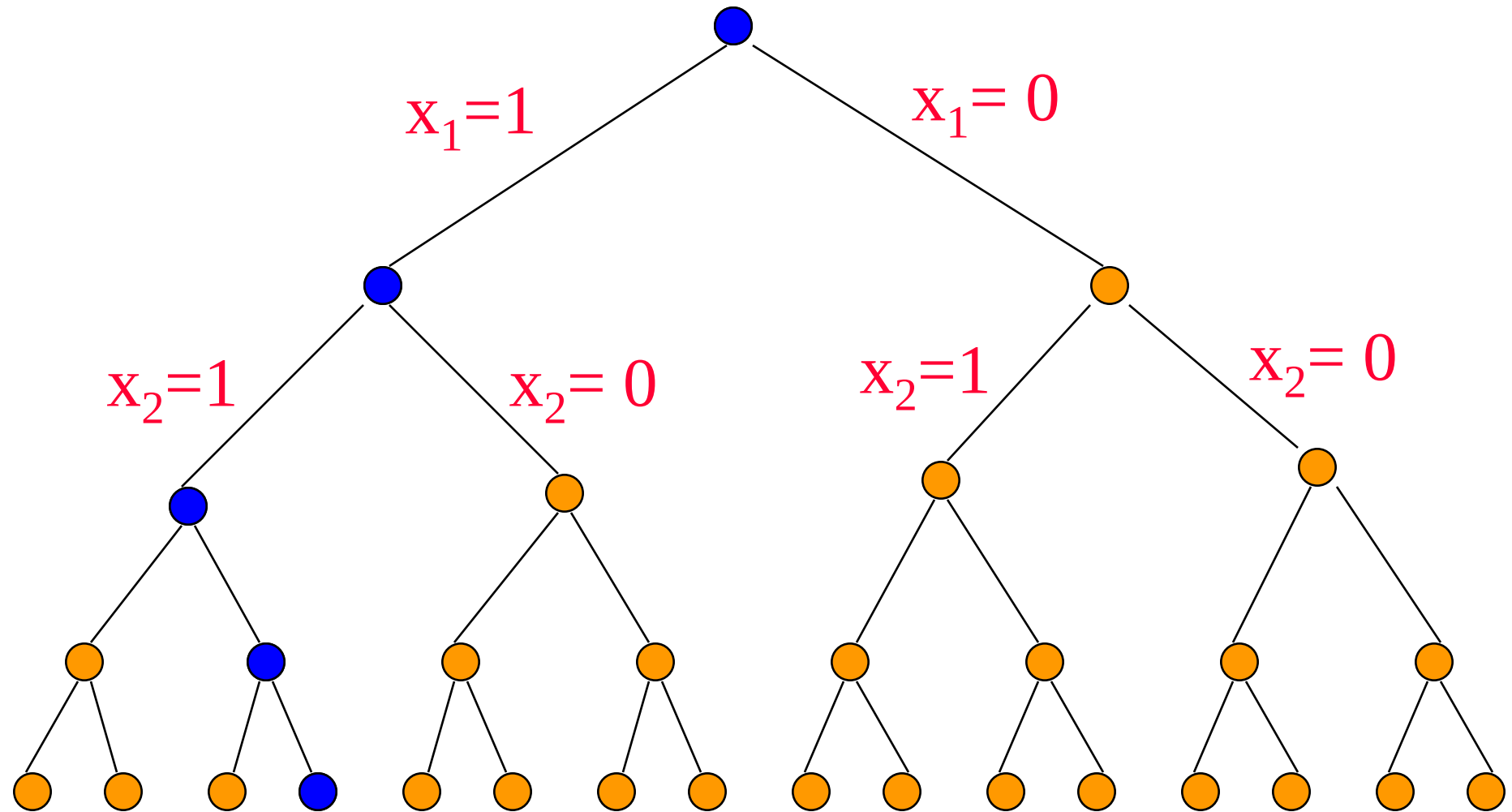
Backtracking Depth-First Search



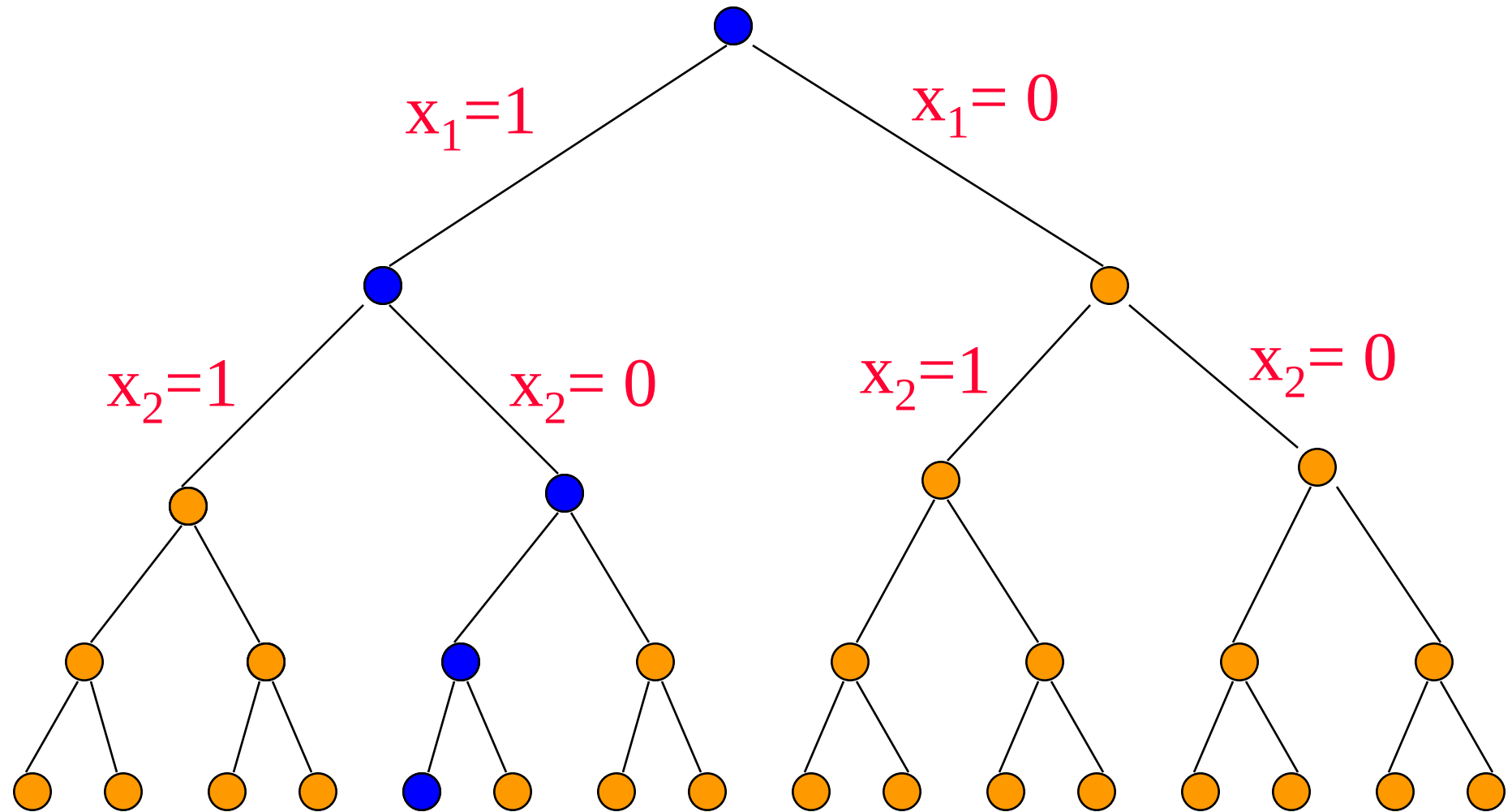
Backtracking Depth-First Search



Backtracking Depth-First Search



Backtracking Depth-First Search



n – queens problem:-

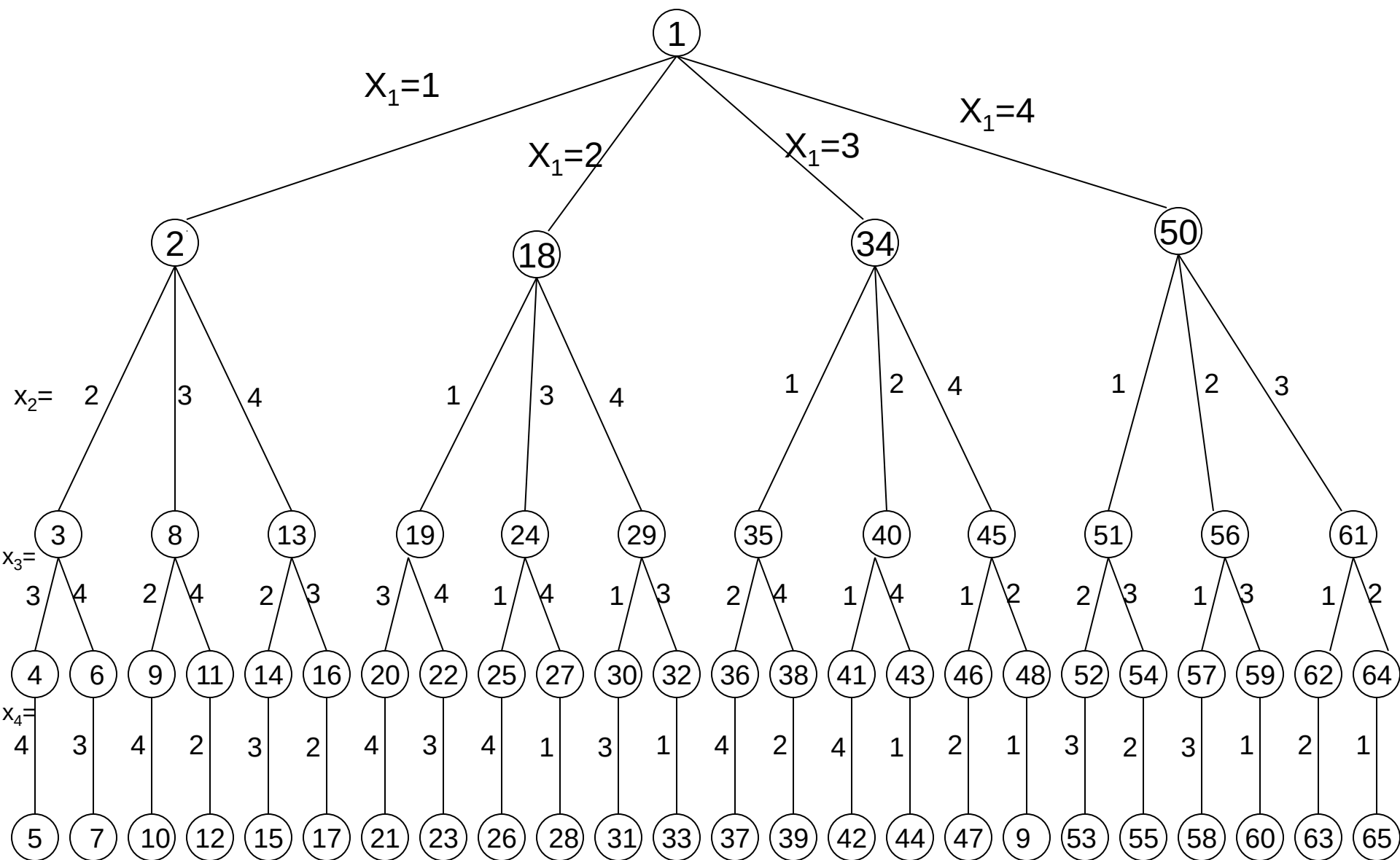
The problem is to place n queens on an $n \times n$ chessboard so that no two “attack” that is no two queens on the same *row, column, or diagonal*.

Defining the problem:-

- Assume rows and columns of chessboard are numbered *1* through *n*.
- Queens also be numbered *1* through *n*.
- Since each queen must be on a different row ,hence assume queen *i* is to be placed on *row i*.
- Therefore all solutions to the *n-queens* problem can be represented as *n-tuples* $(x_1, x_2, \dots, x_n)$, where x_i is the column on which *queen i* is placed.

Tree structure for the case $n=4$.

Note: while constructing solution space tree, to reduce it, do not allow row and column attacks, and allow diagonal attack.



Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

n - queens problem algorithm

- Every element on the same diagonal that runs from the upper left to the lower right has the same *row - column* value.
- Similarly, every element on the on the same diagonal that runs from the upper right to the lower left has the same *row + column* value.

- If two queens are placed at positions (i, j) and (k, l) , then they are on the same diagonal only if

$$i - j = k - l \quad \text{or} \quad i + j = k + l$$

First equation implies

$$j - l = i - k$$

Second equation implies

$$j - l = k - i$$

- Therefore, two queens lie on the same diagonal if and only if $|j - l| = |i - k|$

Algorithm place(k, l)

// It returns true if a queen can be placed in column *l*

// It tests both whether i is distinct from all previous values

// $x[1], \dots, x[k-1]$ and there is other queen on the same

//diagonal.

// Abs(r) returns the absolute value of r.

{

 for j = 1 to k-1 do *// for all previous queens*

 { *// Two in the same column or in the same diagonal*

 if ($x[j] = l$) or ($Abs(x[j] - l) = Abs(j - k)$) then

 return false;

 }

 return true ;

}

Algorithm NQueen(k,n)

//Using backtracking, this procedure prints all
//possible placements of n queens on an $n \times n$
//chessboard so that they are nonattacking.

```
{  
    for  $l = 1$  to  $n$  do      // check place of column for queen  $k$   
    {  
        if place(  $k, l$  ) then  
        {  
             $x[ k ] = l$ ;  
            if(  $k = n$  )then write(  $x[1:n]$  );  
            else NQueens(  $k+1, n$ );  
        }  
    }  
}
```

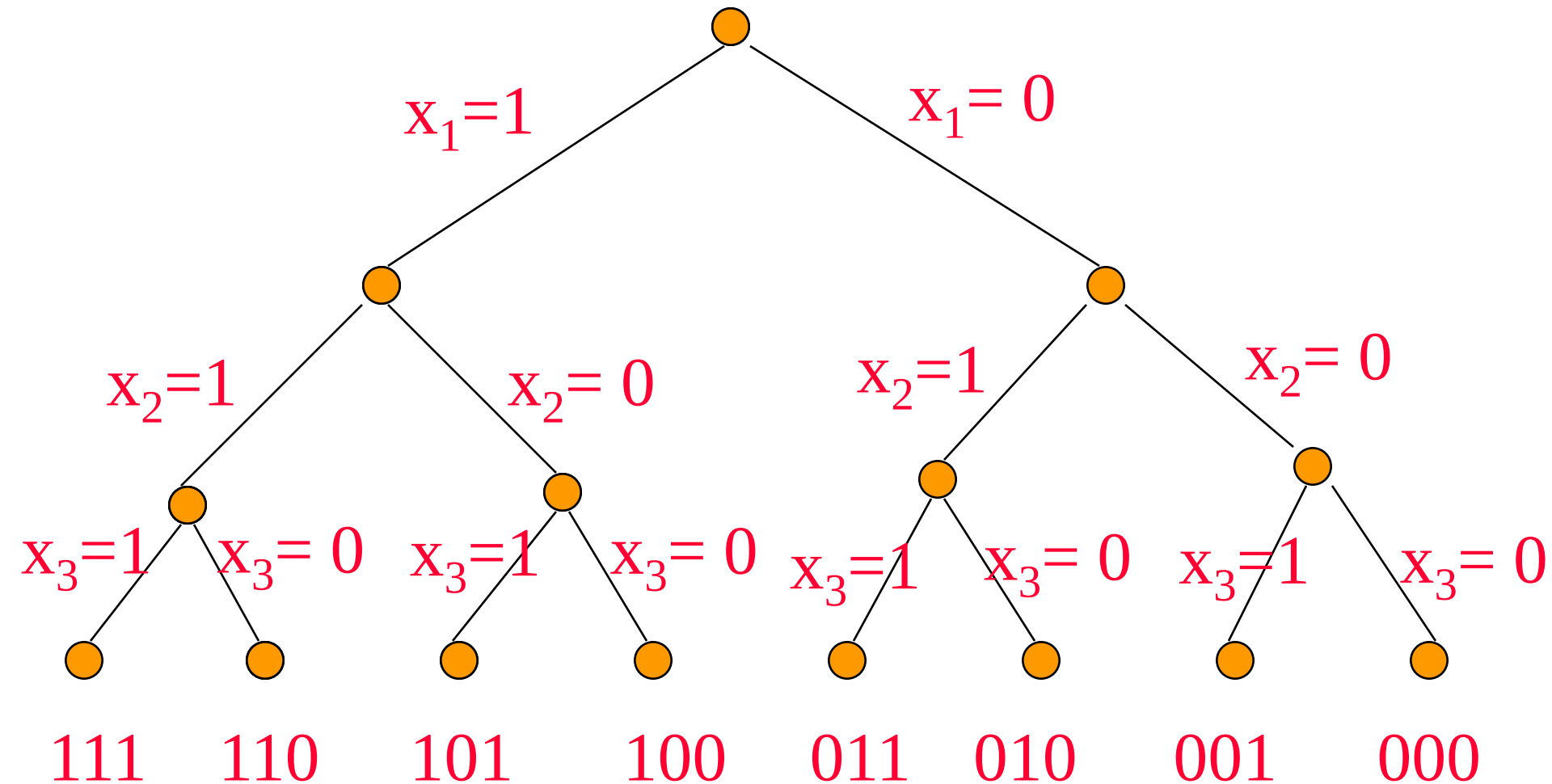

Sum of subsets

- Given n distinct positive numbers w_i , and m , find all subsets that sum to m .
- We can formulate this problem using Fixed sized tuples.

Fixed- sized tuple

- In this method, each solution subset is represented by an n-tuple $(x_1, x_2, \dots, x_n)$.
- $x_i = 0$ if w_i is not chosen and $x_i = 1$ if w_i is chosen

Subset Tree For $n = 3$ (fixed – sized tuple)



Tuples are 111, 110, 101, 100, 011, 010, 001, 000

Algorithm of sum of subsets

- Backtracking solution using *fixed- sized* tuple.
- A simple choice for bounding function is
 $B_k(x_1, x_2, \dots, x_k) = \text{true}$ iff

$$\sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m$$

- Clearly $x_1, x_2, \dots, x_k$ cannot lead to an answer node if this condition is not satisfied.

- The bounding function *strength* further can be increased if we assume the w_i 's in increasing order.
- In this case $x_1, x_2, \dots, x_k$ cannot lead to an answer node if

$$\sum_{i=1}^k w_i x_i + w_{k+1} > m$$

- Therefore, the bounding functions we use are
 $B_k(x_1, x_2, \dots, x_k) = \text{true}$ iff

$$\sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m \quad \text{and} \quad \sum_{i=1}^k w_i x_i + w_{k+1} \leq m$$

The initial call is SumOfSub(0, 1, $\sum_{i=1}^n w_i$)

Algorithm SumOfSub(s, k, r)

// Find all subsets of w[1:n] that sum to m

// It is assumed that $w[1] \leq m$ and $\sum_{i=1}^n w_i \geq m$

// The values of x[j], $1 \leq j < k$, have already been determined.

// $s = \sum_{j=1}^{K-1} w[j] * x[j]$ and $r = \sum_{j=k}^n w[j]$. W[j]'s in increasing order.

{

 x[k]=1; *// left child*

 if(s + w[k] = m) then write(x[1: k]) ; *// Subset found*

 else if (s + w [k] + w[k+1] $\leq$ m)

 then SumOfSub(s+ w[k], k+1, r- w[k])

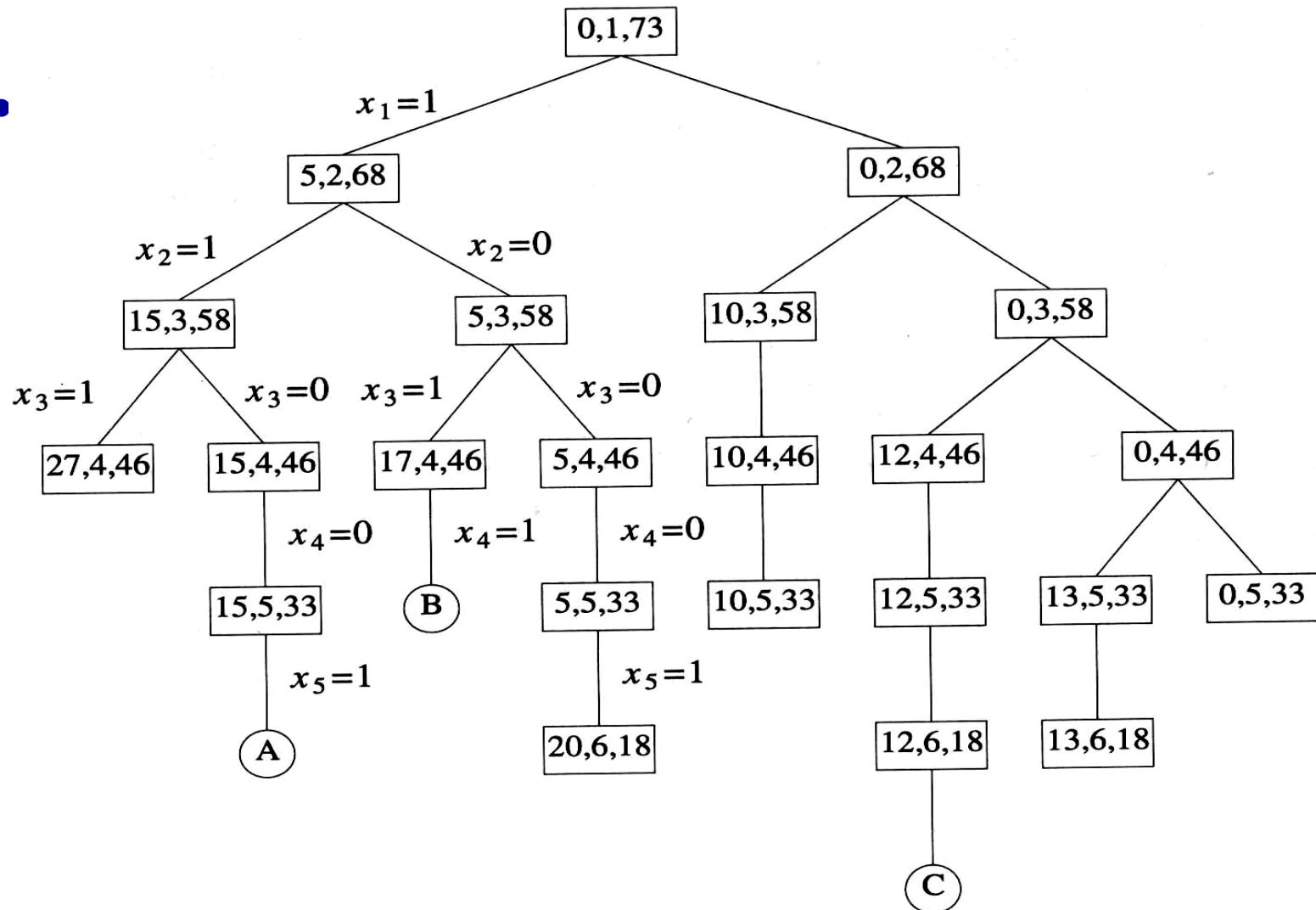
// Generate right child and evaluate B_k

```
if ( ( s + r - w[ k ] ≥ m ) and ( s + w[ k+1 ] ≤ m ) ) then
    {
        x[ k ] = 0;
    }
    SumOfSub( s, k+1, r- w[k] ) ;
}
```

Ex:- $n=6$, $m=30$, $w [1:6] = \{ 5,10,12,13,15,18 \}$

Portion of state space tree generated by SumOfSub.

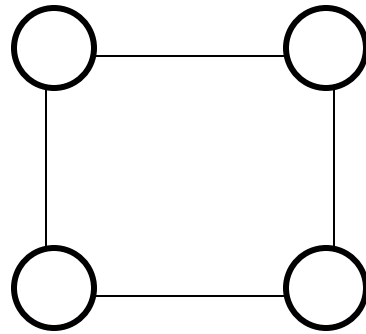
circular nodes indicate subsets with sums equal to m .



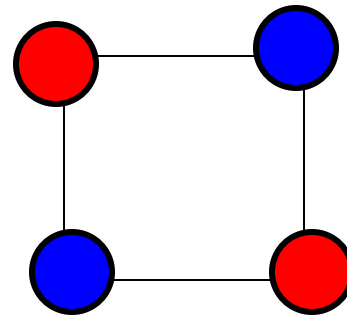
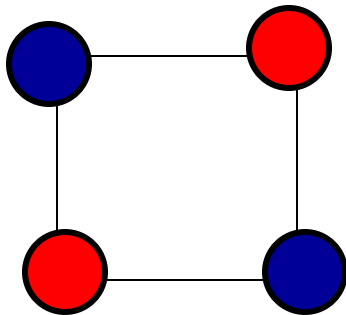
Graph Coloring Problem

- Assign colors to the vertices of a graph so that no adjacent vertices share the same color.
 - Vertices i , j are adjacent if there is an edge from vertex i to vertex j .

Example

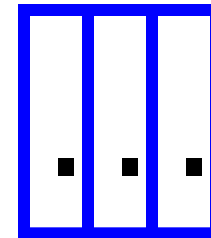
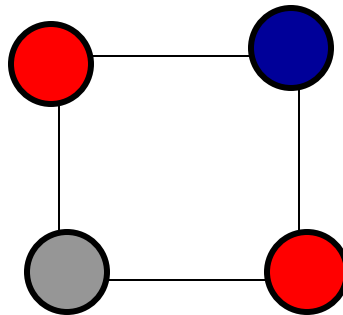
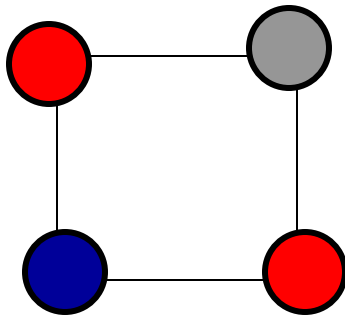
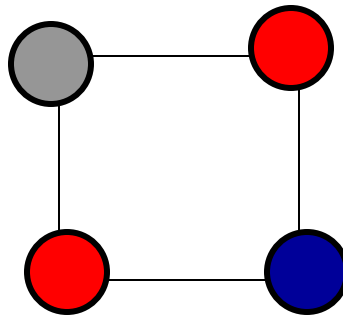
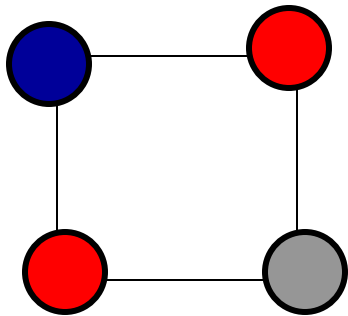


No.of Colors used: 2



Number of Possible ways : 2

No.of Colors used: 3



Some possible ways

- *M-colorability optimization* problem is the smallest integer m (no. colors) with which the graph can be colored.
- This smallest number is called *chromatic* number.

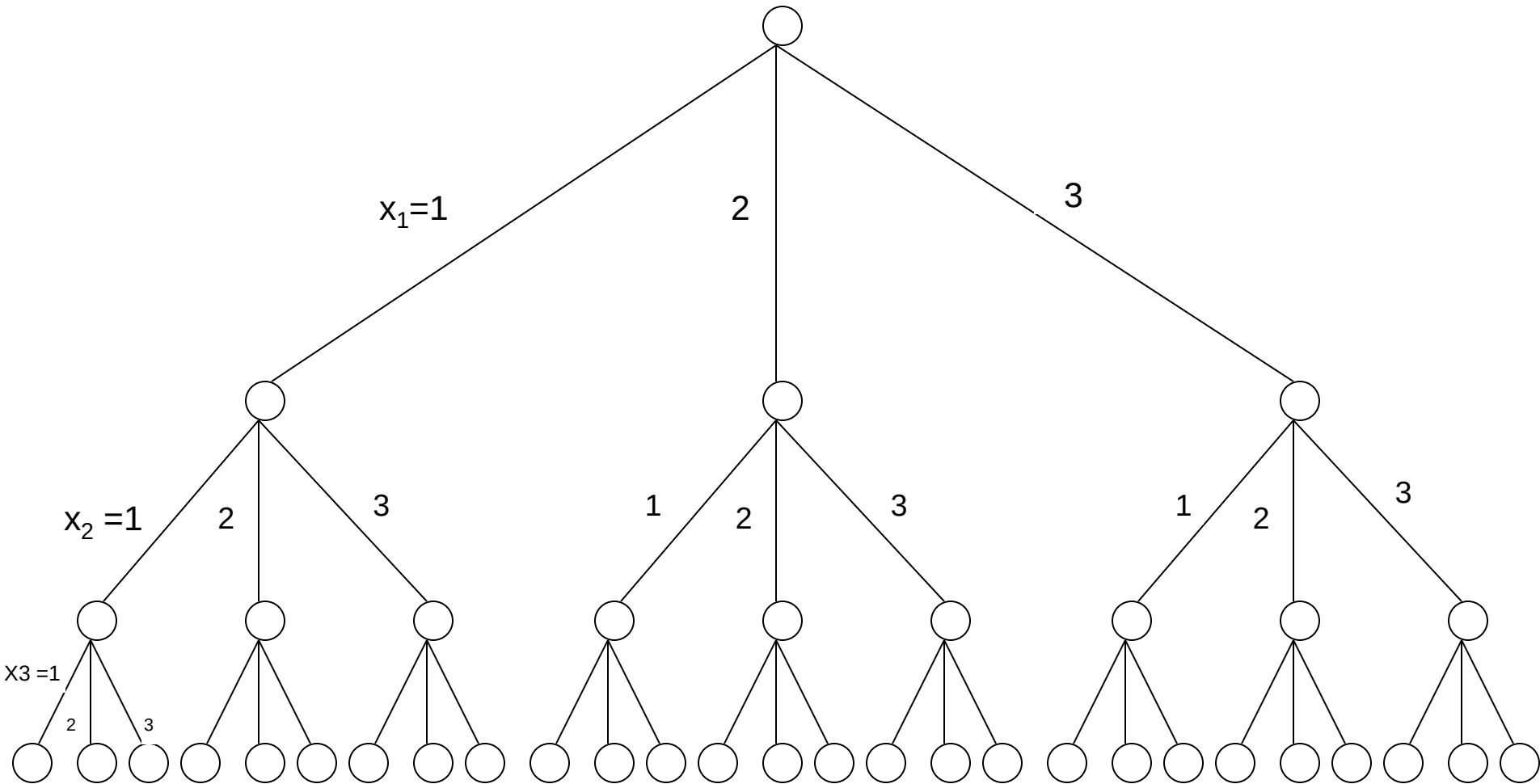
m-colorings problem:-

- Find all ways to color a graph with at *most m* colors.

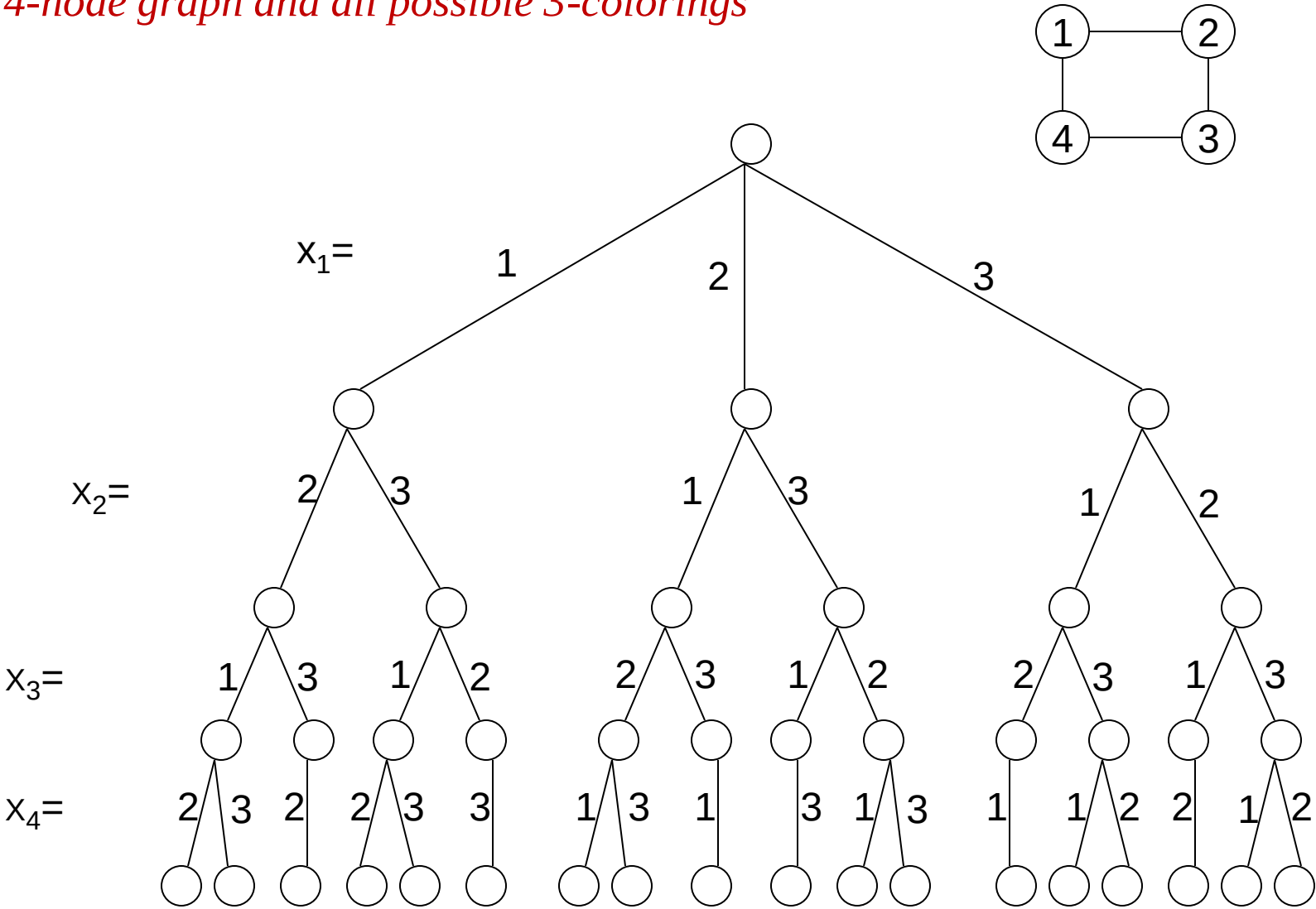
Problem formulation:-

- Represent the graph with adjacency matrix $G[1:n,1:n]$.
- The colors are represented by integer numbers $1, 2, \dots, m$.
- Solution is represented by n -tuple $(x_1, \dots, x_n)$, where x_i is the color of node i .

Solution space tree for mColoring when $n=3$ and $m=3$



A 4-node graph and all possible 3-colorings



Algorithm :- finding all m - colorings of a graph.

Function `mColoring` is called by first assigning the graph to its adjacency matrix, setting the array `x[ ]` to zero, and then calling the statement `mColoring( 1 )`;

Algorithm `mColoring( k )`

// k is the index of the next vertex to color.

```
{  
    repeat  
    {        // Generate all legal assignments for  $x[k]$   
  
        NextValue(  $k$  );           // Assign to  $x[k]$  a legal color  
  
        if (  $x[k]=0$  ) then return; // No new color possible  
  
        if (  $k=n$  ) then // At most  $m$  colors have been used to color the  $n$  vertices  
            write(  $x[1:n]$  );  
        else mColoring(  $k+1$  );  
    } until ( false );  
}
```


Algorithm NextValue(k)

// $x[1], \dots, x[k-1]$ have been assigned integer values in the range $[1, m]$.

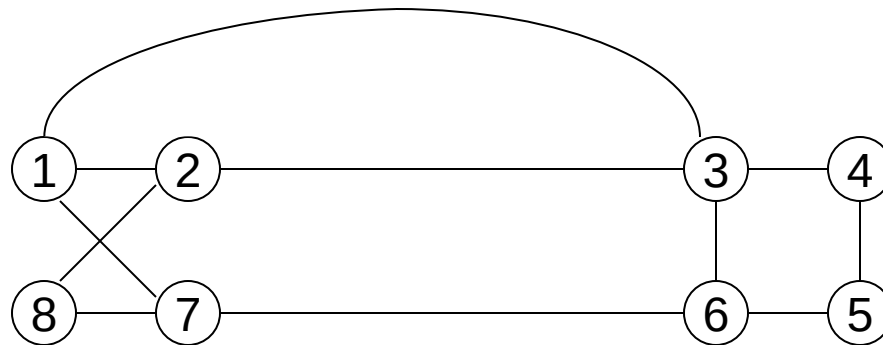
// A value for $x[k]$ is determined in the range $[0, m]$

```
{
  repeat
  {
     $x[k] = (x[k] + 1) \bmod (m + 1);$            // Next highest color.
    if (  $x[k] = 0$  ) then return;           // All colors have been used.

    for j = 1 to n do
    {
      if ( (  $G[k, j] \neq 0$  ) and (  $x[k] = x[j]$  ) ) then
        break;
    }
    if( j = n+1 ) then return; // Color found
  } until ( false );
}
```

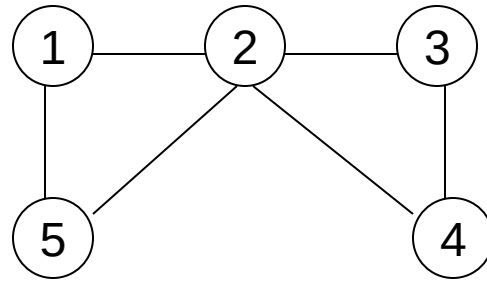
Hamiltonian cycles:-

A Hamiltonian cycle is a *round-trip path along n edges* of connected undirected graph G that visits every *vertex once* and returns to its *starting* position.



The above graph contains the Hamiltonian cycle

1,2,8,7,6,5,4,3,1



The above graph does not contain Hamiltonian cycles.

Note : Traveling sales person problem is a minimum weight Hamiltonian cycle.

Find all possible Hamiltonian cycles

Problem formulation:-

- Represent the graph with adjacency matrix $G[1:n,1:n]$.
- Solution is represented by n -tuple $(x_1, \dots, x_n)$, where x_i represents the i^{th} visited vertex of the cycle.
- Start by setting $x[2:n]$ to zero and $x[1]=1$, and then executing $\text{Hamiltonian}(2)$;

Algorithm Hamiltonian(k)

```
{
  repeat
  {    // Generate values for x[k]

    NextValue( k ); // Assign a legal next value to x[ k ]

    if ( x[k]=0 ) then return; // No new value possible

    if ( k=n ) then write( x[1:n] );
    else Hamiltonian( k+1);
  } until ( false );
}
```

Algorithm NextValue(k)

```
{  
  repeat  
  {  
     $x[k] = (x[k] + 1) \bmod (n + 1);$            // Next vertex.  
    if (  $x[k] = 0$  ) then return;  
    if(  $G[x[k-1], x[k]] \neq 0$  )  
    {  
      for j:= 1 to k-1 do if (  $x[j] = x[k]$  ) then break;  
                           // Check for distinctness  
      if(  $j = k$  ) then // if true, then the vertex is distinct  
        if( (  $k < n$  ) or ( (  $k = n$  ) and  $G[x[n], x[1]] \neq 0$  )  
          then return;  
    }  
  } until ( false );  
}
```

General Backtracking Algorithms :-

- Assume all answer nodes are to be found.
- Let $(x_1, x_2, \dots, x_i)$ be a path from the root to a node x_i in a state space tree .
- Let $T(x_1, x_2, \dots, x_i)$ be the set of all possible paths for x_{i+1} such that $(x_1, x_2, \dots, x_{i+1})$ is also a path to a problem state.
- Assume bounding function B_{i+1} (implicit conditions) such that if $B_{i+1}(x_1, x_2, \dots, x_{i+1})$ is false for a path $(x_1, x_2, \dots, x_{i+1})$, then the path cannot be expanded to reach an answer node.

General Recursive backtracking Algorithm:-

Algorithm Backtrack(k)

// The first k-1 values have been assigned.

// x[] and n are global.

```
{  
    for ( each x[k] ∈ T(x[1],.....x[k-1] )  
    {  
        if( Bk( x[1],x[2],.....x[k] ≠ 0) then  
        {  
            if(x[1],x[2],.....x[k] is a path to answer node )  
                then write( x[1:k] );  
            if ( k < n) then Backtrack( k+1 );  
        }  
    }  
}
```